

Design and Verification of a Simplified AES Encryption Engine

Submitted by: Shreya Sen
Student ID: 231001002086

September 2026

Introduction

The Simplified Advanced Encryption Standard (S-AES) is a compact version of the AES encryption algorithm designed for educational purposes. It operates on a 16-bit plaintext block using a 16-bit key and demonstrates the fundamental concepts of a Substitution-Permutation Network (SPN)

In this project, an S-AES encryption engine was designed and implemented using Verilog HDL for the Nexys 4 DDR FPGA development board. The design implements the complete encryption flow consisting of key expansion, pre-round key addition, two encryption rounds, and ciphertext generation.

Overall Work Flow

The overall encryption flow of the S-AES encryption engine is shown in Figure 1. The plaintext is first combined with the initial round key K_0 . The resulting state then passes through the transformations of the first round. In the final round, the MixColumns operation is omitted, and the resulting state after AddRoundKey with K_2 forms the ciphertext.

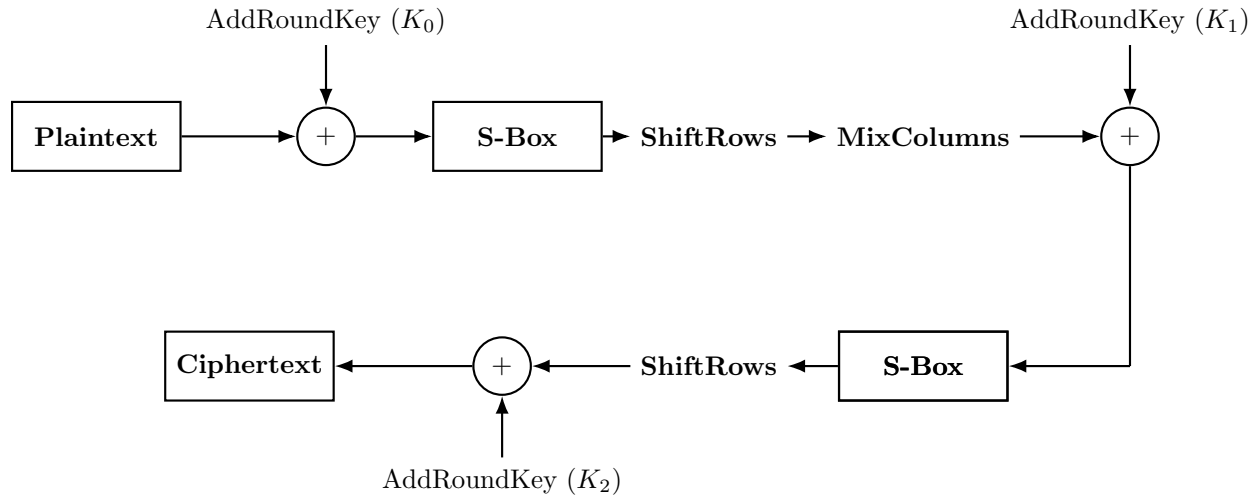


Figure 1: Overall data flow of the S-AES encryption process.

RTL Designing and Working

0.1 Key_Expansion module:

This module generates the three 16-bit round keys K_0 , K_1 , and K_2 from the 16-bit master key. The two round constants used in the implementation are $Rcon1 = 8'h80$ and $Rcon2 = 8'h30$. For the implemented S-AES configuration, the master key is 2D65, resulting in

$$K_0 = 2D65, \quad K_1 = B5D0, \quad K_2 = 1BCB.$$

0.2 S_Box module:

The S-Box was implemented as a 16×4-bit ROM using dedicated BRAM resources rather than general-purpose LUTs. The ROM was initialized with the specified S-AES substitution values (shown in Table 1) using 'sbox.coe' file and integrated using the Vivado Block Memory Generator.

Table 1: S-Box Lookup Table

Input	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Output	9	4	A	B	D	1	8	5	6	2	0	3	C	E	F	7

Since the 16-bit state consists of four 4-bit nibbles, the S-Box module performs four parallel lookups to substitute the complete state in a single S-Box operation. The four lookups are managed internally within the single top-level S-Box module, satisfying the required architecture.

0.3 Shift_Rows module:

This module performs a fixed permutation of the nibble positions as shown below

Initial Position	N_{00}	N_{01}	N_{10}	N_{11}
Final Position	N_{00}	N_{01}	N_{11}	N_{10}

0.4 Mix_Columns module:

This module provides further diffusion by mixing data within each column using the following operations,

$$N'_1 = N_1 \oplus (4 \cdot N_2)$$

$$N'_2 = (4 \cdot N_1) \oplus N_2$$

The multiplication by 4 required in the MixColumns transformation is implemented using a combinational lookup table. The corresponding mapping for all possible 4-bit input values is shown in Table 2.

Table 2: "Multiply by 4" Lookup Table

Input	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Output	0	4	8	C	3	7	B	F	6	2	E	A	5	1	9	D

0.5 Add_Round_Key module:

This module mixes the secret key (K_1 or K_2) with the data state using XOR operation. It is the only step that uses the round keys.

0.6 Top module: SAES

The SAES module serves as the top-level module of the encryption engine and integrates all the individual S-AES components discussed earlier.

A finite state machine (FSM) is used to control the overall encryption sequence. When a new plaintext is detected, it is first latched into `sw_latched` and the key expansion process is initiated. The generated round keys K_0 , K_1 , and K_2 are then used during the encryption process. During the first round, the latched plaintext is first XORed with K_0 , followed by S-Box substitution, ShiftRows, MixColumns, and AddRoundKey with K_1 . In the second round, the result is again passed through the S-Box and ShiftRows operations, followed by AddRoundKey with K_2 and the generated ciphertext is stored in DataReg. Then finally The ciphertext is transferred to the 16-bit led output for display on the FPGA board.

Testbench and Verification

A dedicated Verilog testbench, `tb_saes`, was developed to verify the functionality of the S-AES encryption engine. Self-checking tasks are used to automatically compare the generated ciphertext against known S-AES test vectors. The testbench verifies the plaintexts `0xA06E`, `0x0000`, and `0x6F6B`, with their corresponding expected ciphertexts `0x595A`, `0x0581`, and `0x2590`, respectively. A "PASS" message is displayed when the values match, while a "FAIL" message is displayed along with the expected and actual values when they differ.

```
-----  
PASS  
Plaintext : a06e  
Expected  : 595a  
Actual    : 595a  
-----  
  
PASS  
Plaintext : 0000  
Expected  : 0581  
Actual    : 0581  
-----  
  
PASS  
Plaintext : 6f6b  
Expected  : 2590  
Actual    : 2590  
-----
```

Figure 2: Simulation Output Result

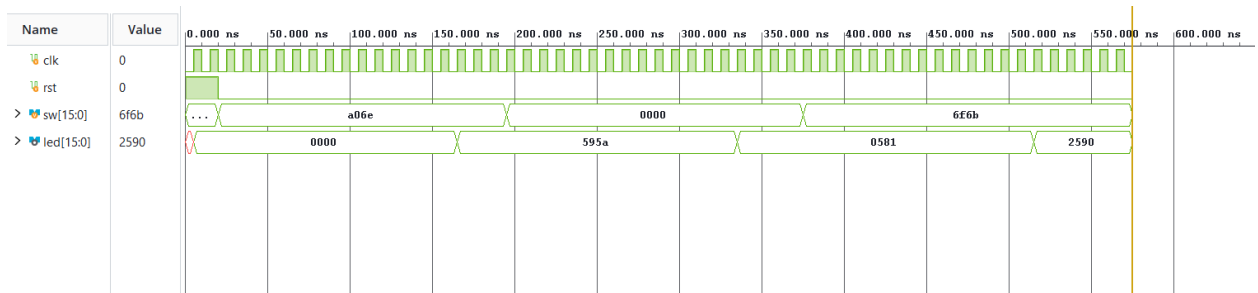


Figure 3: Simulation Output Waveform

Analysis and Results

0.1 BRAM vs. LUT-Based S-Box:

A LUT (Lookup Table) is a fundamental building block that is used to implement logic functions within the FPGA fabric. The LUT can be programmed or configured to represent any desired logic function by specifying the outputs for all possible input combinations.

On the other hand Block RAMs (or BRAM) are used for storing large amounts of data inside of an FPGA.

The S-Box contains a fixed substitution table, where each 4-bit input always maps to a predetermined 4-bit output. Since its contents are fixed and do not depend on any runtime combinational computation, it can be efficiently implemented as a ROM using dedicated BRAM resources. So using BRAM for the S-Box avoids consuming LUT resources and these LUTs can instead be used by other parts of the design, such as the combinational logic associated with the FSM control, XOR operations, multiplexing, and other datapath logics.

0.2 Resource Utilization:

The resource utilization of the implemented S-AES design, obtained from the Vivado Utilization Report, is summarized in Table 3

Table 3: Resource Utilization Table

Site Type	Used	Available	Util%
LUTs	73	63400	0.12
Register as Flip Flop	103	12680	0.08
BRAM tiles	3	135	2.22

0.3 Critical Path:

According to the Vivado Timing Report, the critical path of the design is from the clock input of the S-Box BRAM instance SB5 to the data input of the `sbox_in.reg[2]` flip-flop.

The design has a Worst Negative Slack (WNS) of 5.218 ns and the critical path has a data path delay of approximately 4.747 ns.